

```
function initApp() {  
  const config = {  
    type: 'vanilla'  
  };  
  buildUI(config);  
}  
  
function initApp() {  
  const config = {  
    type: 'vanilla'  
  };  
  
  export Component Vte {  
    // classe there items  
    buildUI(config);  
  }  
}  
  
class Component extends View {  
  render() {  
    return createNode('div', {  
      className: 'container'  
    });  
  }  
}
```



PROGRAMAÇÃO FRONT-END 2 (PFE2)

Da Codificação Artesanal à Engenharia de Software com IA.

A Fundação

O Sistema Nervoso da Web

Manipulação do DOM

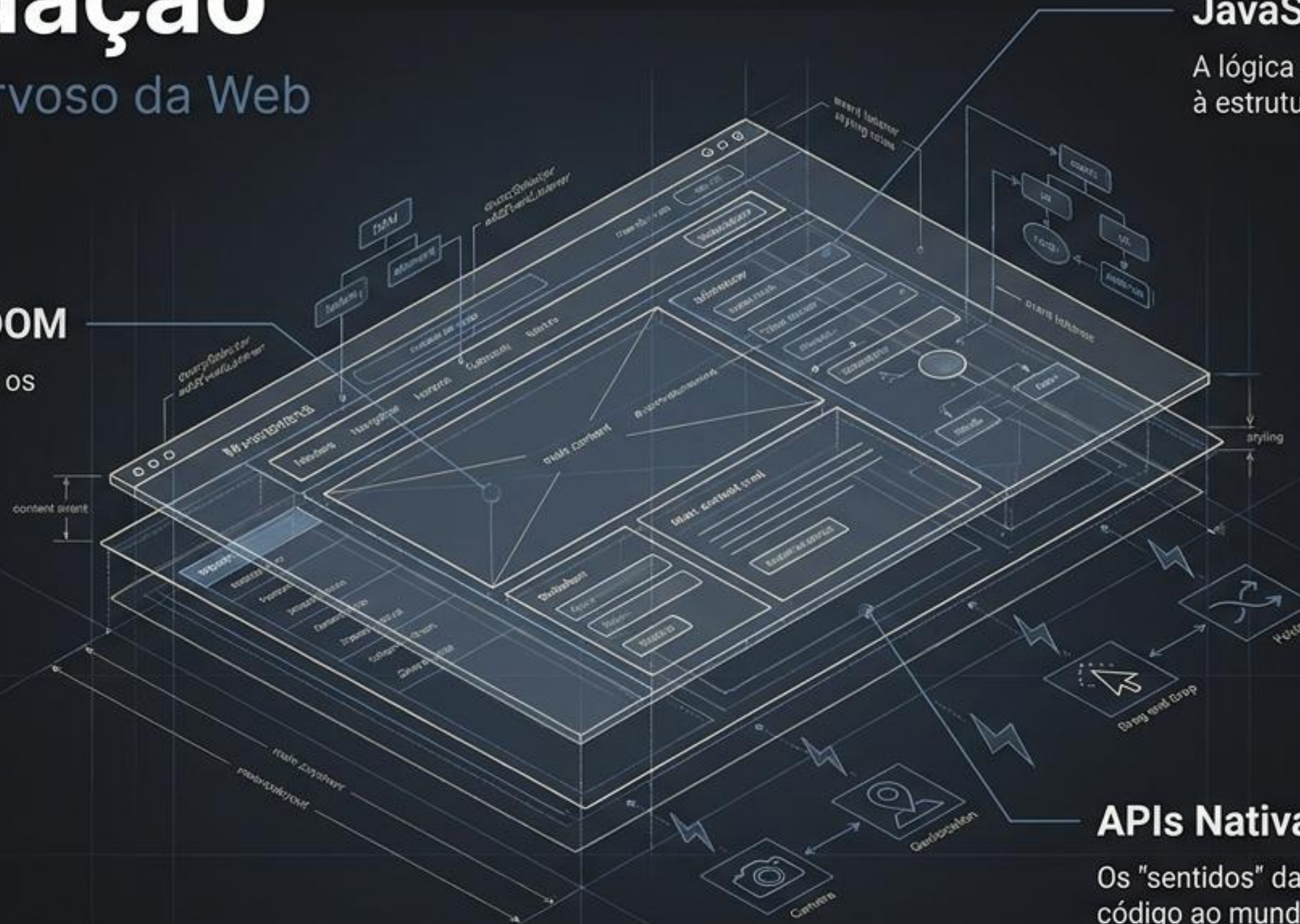
O controle absoluto sobre os nós e eventos da página (querySelector, addEventListener).

JavaScript ES6+

A lógica que dá vida e movimento à estrutura estática.

APIs Nativas & Web Storage

Os "sentidos" da aplicação, conectando o código ao mundo exterior através da Câmera, Geolocalização, Drag & Drop e fetch().

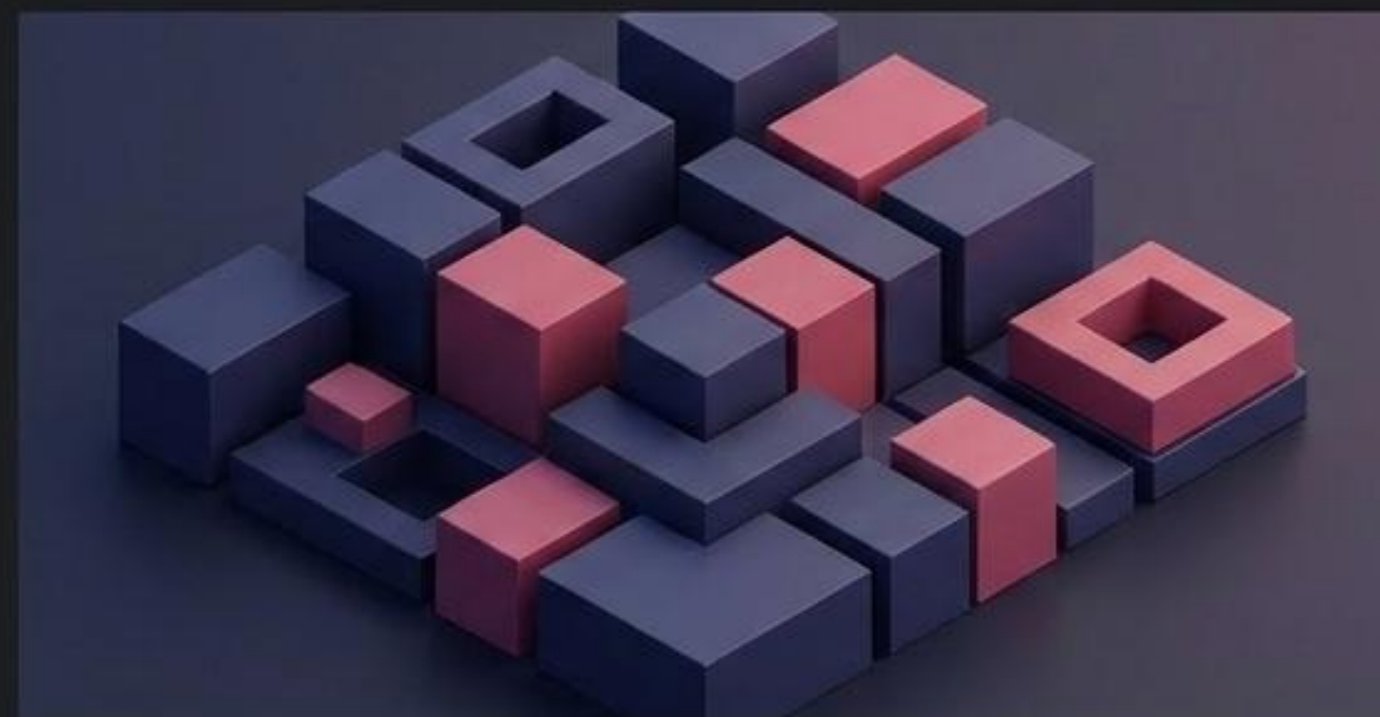


O Novo Paradigma

O Passado (Imperativo)



O Futuro (Declarativo)



O Salto para o React

Pensar a web não como páginas, mas como blocos de construção.

Interfaces Inteligentes

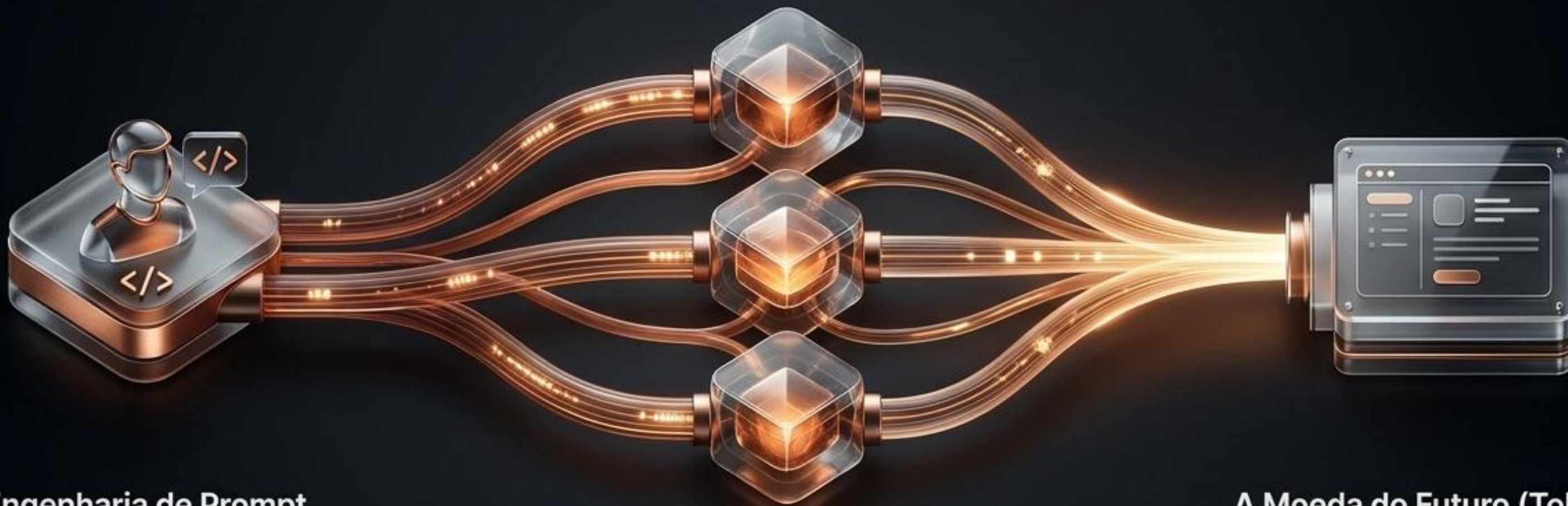
Construção de componentes modulares, reutilizáveis e reativos (JSX, State, Props).

Consumo Contínuo

Integração fluida e em tempo real com APIs externas sem recarregar a página.

O Motor do Futuro

IA Generativa como Multiplicador de Inteligência



Engenharia de Prompt

A nova linguagem de programação — a clareza do seu pedido define a qualidade do software.

Google Antigravity & Vibe Coding

A transição de digitar cada linha para direcionar a intenção do código.

A Moeda do Futuro (Tokens)

Entendendo como gerenciamos, otimizamos e “pagamos” por esse poder computacional massivo.

Tecnologias do Projeto

As Ferramentas para Construir o SENAI Hub

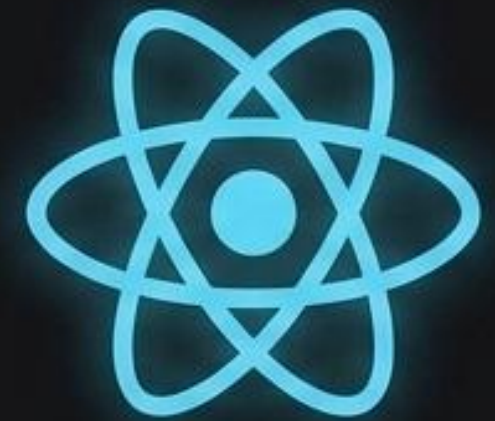
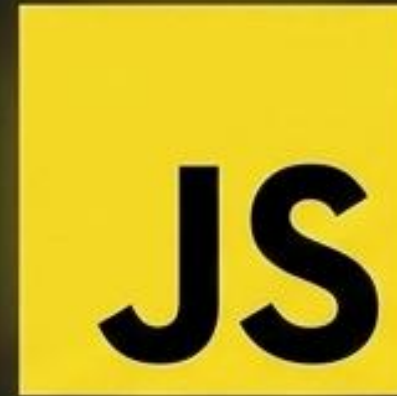
HTML



CSS3



JS



Antigravity



Quem Você Se Tornará? **O Desenvolvedor 10x.**

O engenheiro de software moderno não é aquele que apenas digita código mais rápido. É aquele que alia pensamento analítico, autogestão e inteligência emocional para orquestrar a Inteligência Artificial na construção do futuro.

A jornada começa agora.



O Método de Construção: Aprender Fazendo

O ALICERCE (MSEP):

O conhecimento técnico se consolida na prática real. Não memorizamos código; nós construímos soluções.



O FIO CONDUTOR:

Situações de aprendizagem desafiadoras e contextualizadas com a indústria.

A ABORDAGEM:

O instrutor atua como Gerente de Projeto. Vocês atuam como Desenvolvedores Front-end em um ambiente de agência.



O Mapa da Obra: Nossa Jornada de 20 Semanas



Fase 1: As Fundações (Hoje)

DOM, Eventos e
Interatividade Essencial.

Fase 2: Estruturas Complexas

Design Responsivo, Arrays e
APIs.

Fase 3: O Arranha-Céu (React)

Componentização, Estado, e
Interfaces Declarativas.

Avaliações: Duas entregas práticas (AP1 e AP2) e uma
Avaliação Contínua (AC) baseada no Projeto Integrador.

A Matriz da Tríade Front-end

HTML (A Estrutura)

Papel: Semântica e Marcação.

Analogia: O concreto, o aço e as paredes do edifício.

Saída: Elementos estáticos na tela.



CSS (O Acabamento)

Papel: Estilização e Layout.

Analogia: A pintura, o design de interiores e a fachada.

Saída: Identidade visual e responsividade.



JavaScript (A Lógica)

Papel: Comportamento e Interatividade.

Analogia: A rede elétrica, os elevadores e os sensores.

Saída: Respostas a cliques, validações e dados dinâmicos.



Pilar 3: O Sistema Elétrico da Web

O **JavaScript** é a linguagem que transforma documentos inertes em aplicações vivas.



Controle Absoluto:

Define o que acontece quando um botão é clicado, como os dados são processados e como a interface reage.

O Nosso Foco:

A partir de hoje, deixamos de apenas desenhar telas e passamos a engenhar comportamentos.

O Projeto Diretor: SENAI Hub

A Situação de Aprendizagem:
Nosso Projeto Integrador.

A Missão:
Construir um sistema completo
de gestão pessoal e acadêmica.

Os Requisitos:
Interfaces interativas, consumo de
APIs externas e transição fluida
para React.



Cada ferramenta que aprendermos será uma peça diretamente aplicada neste projeto.

Visão de Raio-X: Decompondo a Interface



Camada de Entrada:

```
<input id='entrada-nome'>
```

Camada de Ação:

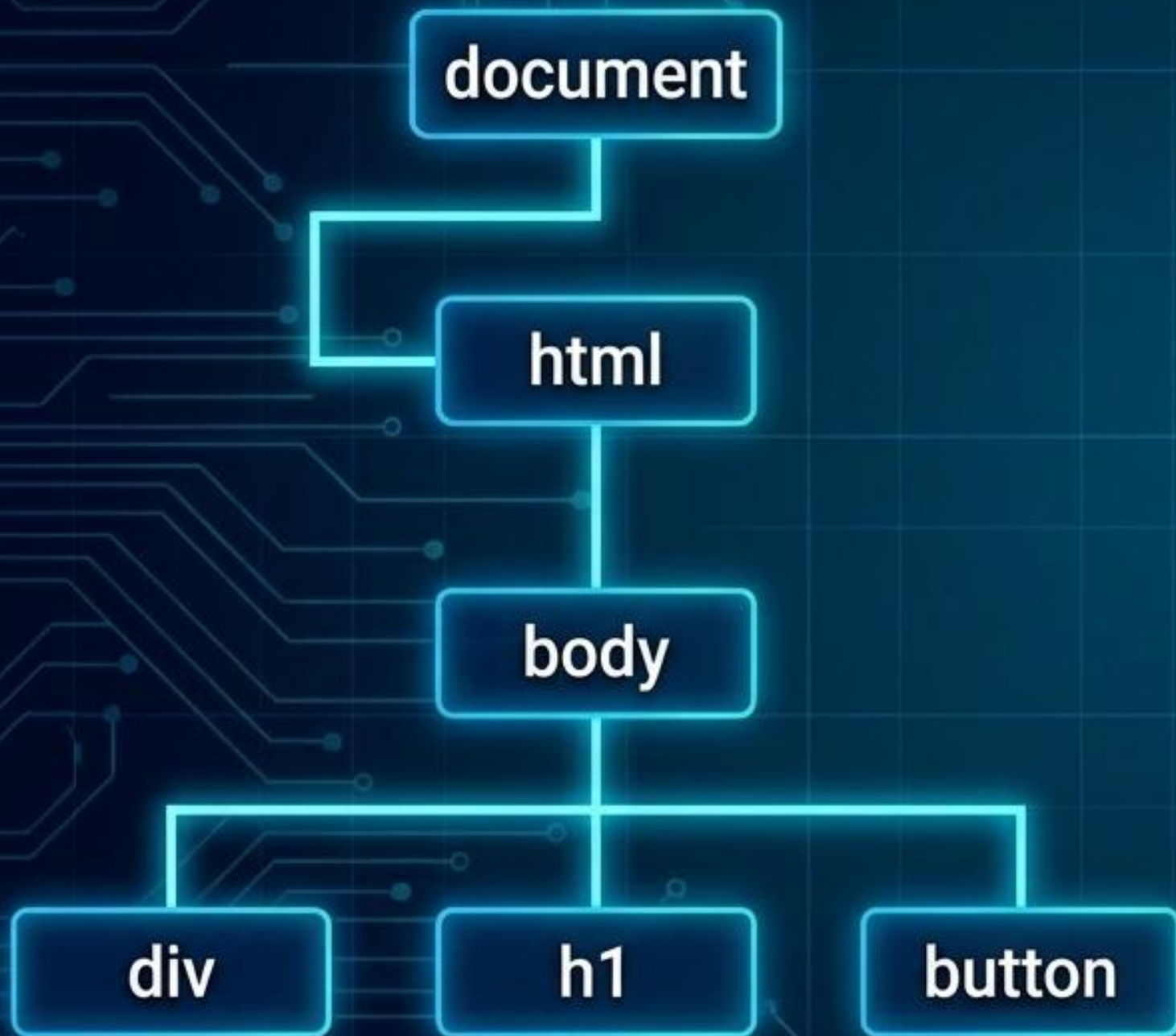
```
<button class='btn-primario'>
```

Camada de Exibição:

```
<ul id='lista-tarefas'>
```

Pensar como um Front-end é olhar para o produto final e enxergar a matriz de elementos individuais.

A Planta Baixa Digital: O Document Object Model



O DOM não é o seu código HTML. É a representação em memória que o navegador cria ao ler o seu HTML.

É uma API (Interface de Programação de Aplicação) que permite ao JavaScript conectar-se à página.

Ao manipularmos a planta baixa (o DOM), o navegador redesenha o edifício (a tela) em tempo real.

O Cinto de Utilidades do DOM

**A Ferramenta 1:
O Olho (Seletores)**



Como o JS encontra
um elemento na tela.

**A Ferramenta 2:
O Ouvido (Ouvintes)**



Como o JS percebe a
ação do usuário.

**A Ferramenta 3:
As Mãos (Modificadores)**



Como o JS altera o
estado da página.

**Dominar o Front-end requer o uso coordenado
destas três ferramentas.**

O Olho: Localizando Elementos na Planta

`document.querySelector()`

Utiliza a mesma sintaxe poderosa que você já conhece do CSS.

HTML

```
<button id='meu-botao'>  
  Salvar</button>
```

JS

```
const botao =  
document.querySelector(  
  '#meu-botao');
```

Por ID: `document.querySelector('#meu-botao');`

Por Classe: `document.querySelector('.painel-alerta');`

Por Tag: `document.querySelector('h1');`

O Ouvido e o Cérebro: Escutando e Reagindo



Comando: `elemento.addEventListener('tipo', função)`
O navegador fica permanentemente escutando uma ação específica naquele elemento.

Arrow Functions (`=>`):
A sintaxe moderna, limpa e direta para escrever a função de reação.

```
botao.addEventListener('click', () => {  
    /* Lógica de reação aqui */  
});
```


O Circuito Fechado: Ciclo do Event Listener



As Mãos (Dados): Lendo e Escrevendo

Lendo a entrada do usuário: **.value**

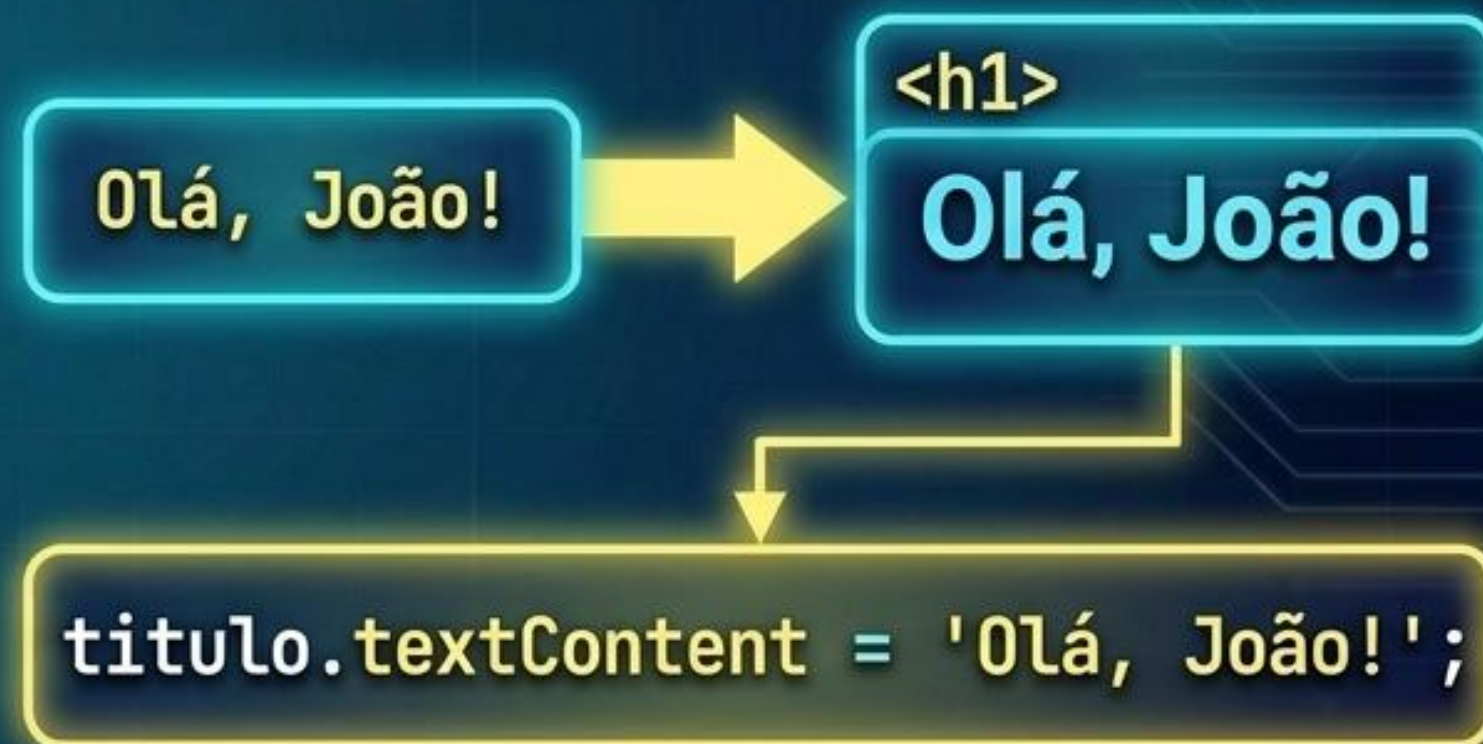
Captura o que foi digitado em formulários e inputs.



O valor é lido do campo.

Escrevendo na tela: **.textContent**

Injeta ou altera o texto puro dentro de um elemento HTML.

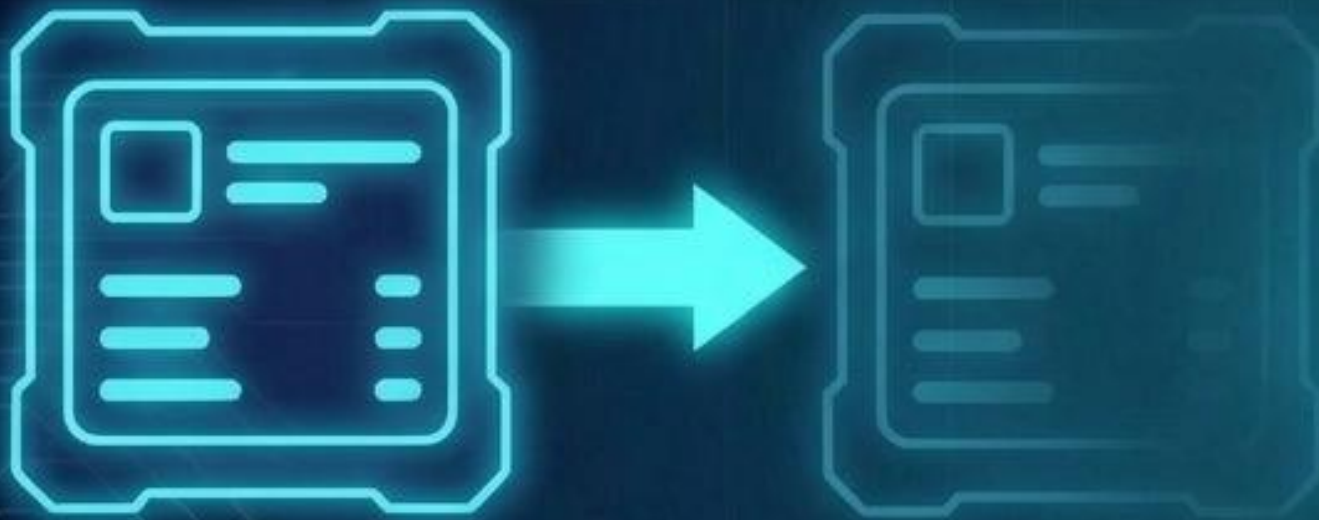


O texto é escrito no elemento.

As Mãos (Visual): Alterando Estados

Escondendo e Revelando: `style.display`

Altera o CSS inline diretamente.



```
caixa.style.display = 'none'; (Desaparece)  
caixa.style.display = 'block'; (Aparece)
```

A Magia do Toggle: `classList.toggle()`

Adiciona uma classe CSS se não existir, ou remove se já existir.



```
painel.classList.toggle('dark-mode');
```


Matriz de Estado do Elemento (Antes e Depois)

Comando JS	Estado Anterior (UI)	Estado Atualizado (UI)
<code>.textContent = 'Sucesso!'</code>		
<code>.value (Lendo)</code>		
<code>.style.display = 'none'</code>		
<code>.classList.toggle('ativo')</code>		

O Acabamento Moderno: **Template Literals**

- ⚙️ A arte de misturar texto fixo com variáveis dinâmicas.
- ⚙️ Usamos **crases** (backticks) `` em vez de aspas.
- ⚙️ As variáveis são injetadas diretamente usando **`${variavel}`**.

~~titulo.textContent = 'Bem-vindo, ' + nomeUsuario + '!';~~

A Solução Elegante:

titulo.textContent = `Bem-vindo, \${nomeUsuario}!`;

A Sala de Reuniões: Dinâmica dos Laboratórios

A partir de agora, a teoria acabou. Estamos em ambiente de agência.

Meu Papel (Cliente/Gerente)

Darei os requisitos de negócios, fornecerei os arquivos base HTML/CSS e validarei as entregas finais.

Não escreverei o código por vocês.

Seu Papel (Desenvolvedor Front-end)

Traduzir os requisitos abstratos em lógica JavaScript funcional.

Utilizar o Cinto de Utilidades do DOM para resolver os problemas reais.

Laboratório 1: Boas-Vindas Dinâmico (Guiado)



Objetivo Claro

A diagram of a web browser window showing a login form. It has a text input field labeled "Nome de Usuário" and a blue button labeled "Login".

A diagram of a web browser window showing the result of a successful login. The text "Bem-vindo, Ana!" is displayed in the center of the page.

Modalidade:

Prática Guiada (Faremos juntos).

Requisito de Negócio:

O **SENAI Hub** precisa saudar o usuário pelo nome após o login.

A Missão:

1. Selecionar o input, o botão e o título.
2. Escutar o evento de clique no botão.
3. Alterar o `textContent` do título usando **Template Literals**.
4. Bônus: Esconder o campo e o botão usando `style.display = 'none'`.

Laboratório 2: Atualizador de Status (Individual)



Modalidade:

Voo Solo (Mão na Massa Individual).

Requisito de Negócio:

Os usuários precisam atualizar rapidamente seu status de disponibilidade na plataforma.

A Missão:

Criar botões de 'Online' e 'Ocupado'. Ao clicar, alterar o texto de uma tag `<p>` e mudar sua cor através da adição e remoção de classes CSS específicas.

Dica do Gerente: Lembre-se de limpar a classe CSS anterior antes de adicionar a nova classe de cor!

Laboratório 3: Painel de Controle e Modo Noturno



Modalidade:

Pair Programming (Em Duplas).

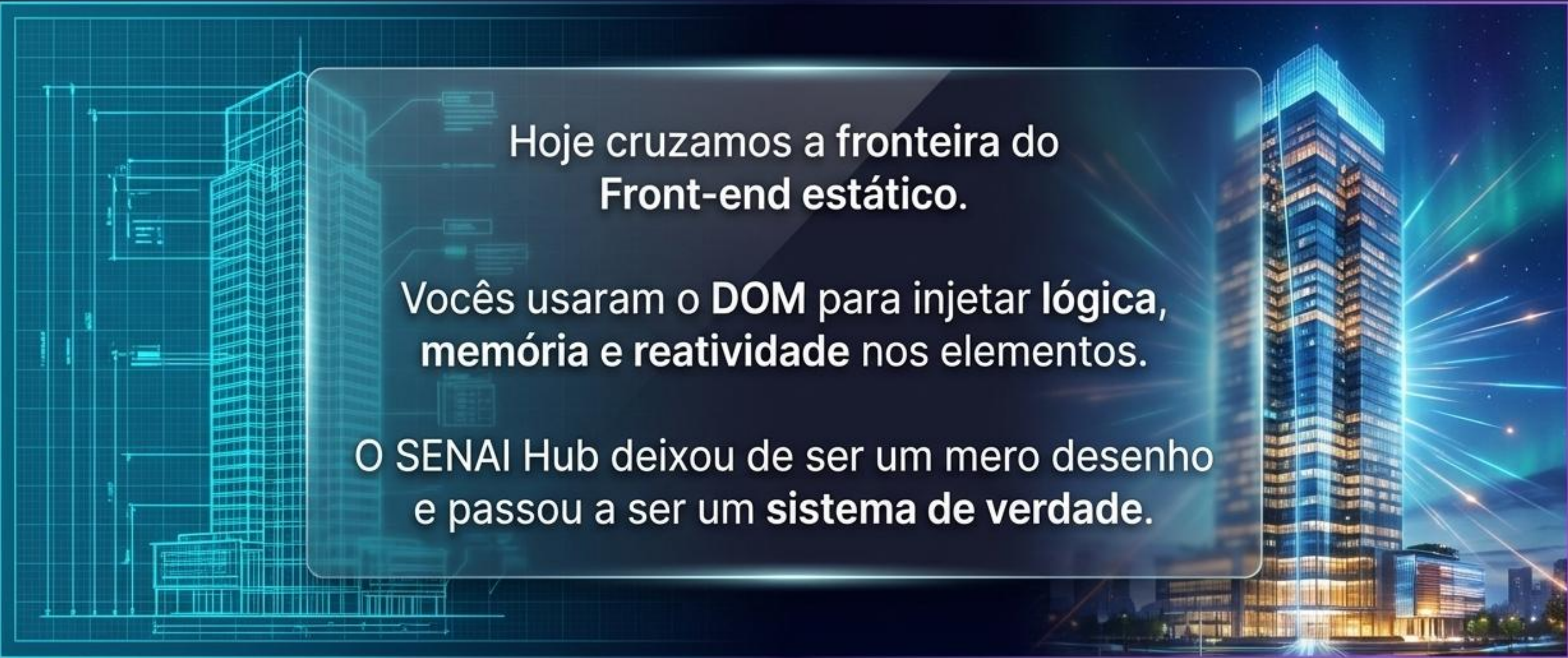
Requisito de Negócio:

A interface principal deve ser confortável para uso noturno prolongado.

A Missão:

- Utilizar `classList.toggle('dark-theme')` diretamente no elemento `<body>`.
- O botão de controle deve alternar seu próprio texto condicionalmente (ex: de 'Ativar Modo Noturno' para 'Desativar Modo Noturno').

Missão Cumprida: O Prédio Ganhou Vida



Hoje cruzamos a fronteira do
Front-end estático.

Vocês usaram o **DOM** para injetar **lógica, memória e reatividade** nos elementos.

O SENAI Hub deixou de ser um mero desenho e passou a ser um **sistema de verdade.**



Próximo Nível: Preparar estruturas de dados complexas (Arrays e Objetos) para popularmos nossas listas de tarefas automaticamente. O projeto está apenas começando.